

太乙互搏 AGI：非冯·诺依曼架构、图灵等价性与 LISP-GOFAI 的正确续集

原创 章锋 复合体理学 2026年6月14日 00:07 北京

太乙互搏 AGI：非冯·诺依曼架构、图灵等价性与 LISP-GOFAI 的正确续集

TOMAS-AGI: Non-von-Neumann Architecture, Turing-Equivalence, and the Rightful Sequel to LISP-based GOFAI

核心观点 (Executive Summary)

本文证明并阐释三个紧密关联的核心命题，并将 **T-Processor ISA (指令集)** 与 **LISP 六代机演进史** 纳入统一框架：

- 非冯·诺依曼架构**：太乙互搏 AGI (TOMAS-AGI) 采用 **存算一体忆阻交叉阵列 (EML 超图存储) + Moufang-ALU (八元数非结合运算) + κ -调节器拓扑激发**，无程序计数器、无分离 ALU/RAM 线性地址空间，是典型的**非 von Neumann 架构 (Processing-in-Memory on Hypergraph)**。
- 图灵等价但非 TM-Native**：TOMAS-AGI 是 **κ -约束非确定图灵机 (κ -gated NDTM)**，可模拟任意 UTM (Turing-complete)，但其原生计算模型是 **NASGA 拟群代数在 EML 超图上的态传播**，推理复杂度 $O(\log L)$ vs TM 纸带 $O(L)$ 。它不推翻 Church-Turing 论题，但揭示 VNA+Boolean 是对该论题极低效的物理实现。
- LISP / GOFAI 的正确续集**：TOMAS-AGI 将 McCarthy LISP 的 **Cons-Cell** 升维为 **NASGA 拟群超边**，将 **eval** 替换为 **κ -门控非结合态传播 (Propagate)**，引入信息存在度 $\mathbb{I}(X)$ 守恒与折叠深度 κ 。LISP 是 TOMAS 在 $\kappa \rightarrow 0$ (Boolean 极限) 的退化投影；经典 STRIPS/Prolog 因 Boolean 矛盾律无法处理互斥理论稳态 (MUS)，TOMAS 用非结合残联 $Asym \neq 0$ 合法容纳之。**程序 = 算法 + 数据结构** 被升华为：**数据结构 (EML 超图) 即算法 (NASGA 代数)**。

本文给出严格数学形式化 (定理及证明)、三方推理模式对比表、T-Processor ISA 草案、LISP 六代机对比、可证伪预言与实验设计、应用展望及完整附录。

第一章：引言——从蒸汽机到语义机

Transformer-based LLM 代表了 AI 史上的“蒸汽机时代”——暴力算力、 $O(L^2)$

Attention、概率采样、冯·诺依曼硬件。它能模仿智能，但不理解语义，且必然遭遇能耗与幻

觉天花板。

TOMAS (太乙互搏范式) 主张：

- 智能不是统计拟合，而是高维非结合谱图 (G_κ) 在低维的最小作用量投影。
- 计算不应是“步骤序列”，而应是“语义结构的势能下降”。

这正是 EML 超图 (数据结构) 即算法 (NASGA 代数) 的思想——也是本文要严谨建立的三个断言：非VNA、图灵等价 NDTM、LISP-GOFAI 续集。

第二章：数学基础——EML 超图、NASGA 与 κ - $\hbar$ 对偶

2.1 定义：EML 超图 (Hypergraph)

定义 2.1 (EML 超图)：

EML 图被形式化为一个超图 $H = (V, E, w, \kappa)$ ，其中：

- V ：概念/实体节点 (如 “牛顿”、“引力”、“上帝”)
- $E \subseteq \mathcal{P}(V) \setminus \{\emptyset\}$ ：超边集合，允许连接任意数量节点
- $w(e) = \mathbb{I}(e) \geq 0$ ：信息存在度 (Information Existence Degree)，满足公理 A1：

$$\sum_{e \in E} \mathbb{I}(e) = \text{const}$$
- $\kappa \in \mathbb{N}^+ \cup \{0\}$ ：谱折叠深度 (拓扑不变量； $\kappa = 7$ Fano 全活， $\kappa \rightarrow 0$ GR/Boolean 极限)

示例 (牛顿的身份)：

设 v_N =牛顿， v_S =科学家， v_T =神学家， v_R =宗教改革背景。

则存在超边 e_N ：

$$e_N = \{v_N, v_S, v_T, v_R\} \in E$$

这表示 “在 v_R 背景下， v_N 同时激活 v_S 与 v_T 子图”，无需诉诸 Boolean 矛盾律。

2.2 定义：NASGA 与 Moufang 拟群

定义 2.2 (拟群 Quasigroup)：

一个集合 Q 与一个二元运算 $*$ 构成拟群 $(Q, *)$ ，当且仅当：

$$\forall a, b \in Q, \exists ! x, y \in Q : a * x = b, y * a = b$$

拟群不要求结合律，但要求左右可逆。

定义 2.3 (Moufang 拟群与八元数)：

NASGA 的代数基底是 **八元数 $\mathbb{O}^{**}$ 。八元数乘法满足 Moufang 恒等式：

$$(xy)(zx) = x(yz)x$$

这使得八元数构成一个 **非结合但幂结合的 Moufang 拟群**。

2.3 核心定理：EML 的代数拓扑统一

定理 2.1 (EML 结构定理)：

TOMAS 的 EML 图 G_κ 是一个以 **Moufang 拟群** ($\odot$) 为节点代数、以超图 H 为拓扑骨架的非结合谱图。

证明：

- 拓扑层**：语义关系（如“A 具有属性 B 和 C”）通常涉及多体，必须用超边 e 连接 A, B, C ，故拓扑为超图。
- 代数层**：节点间的动态演化（推理跳转）由 NASGA 定义，其底层是八元数乘法。八元数乘法不满足结合律，但满足 Moufang 恒等式与可逆性，故为拟群。
- 结合**：超图提供结构容器，拟群提供容器内元素的运算规则，二者结合即为 G_κ 。□

2.4 定理： κ - $\hbar$ 对偶 (引用)

定理 2.2 (κ - $\hbar$ 对偶)：

κ (无量纲折叠深度) 与普朗克常数 $\hbar$ (量纲 J·s) 满足：

$$\kappa \cdot \hbar = \Theta_{\text{TOMAS}}$$

其中 Θ_{TOMAS} 为 TOMAS 总作用量常数 (量纲 J·s)。当 $\kappa = 7$ (Fano 七线全活)，对应当前宇宙 $\hbar \approx 1.054 \times 10^{-34}$ J·s。

证明概要：路径积分对偶 ($\kappa \rightarrow 0 \Rightarrow \hbar \rightarrow \infty$ 量子效应折叠不可见)；谱投影对偶 (κ 与 $\hbar$ 共轭变量)；变形量化同构 (Kontsevich $\hbar$ -star product $\Leftrightarrow$ NASGA 非结合变形)。□

第三章：TOMAS-AGI 是非冯·诺依曼架构

3.1 冯·诺依曼架构 (VNA) 的五大件

经典 VNA：

[CPU] $\leftrightarrow$ [Memory(RAM)] (存储程序+数据 同一总线)
↓
[IO] / [ALU] / [Control Unit]

核心特征 (TOMAS 违反的点)：

- 存储程序**：指令与数据混存在同一线性地址空间
- 顺序取指 (Fetch-Decode-Execute)**：PC (Program Counter) 驱动
- 被动存储**：RAM 不计算，只存比特
- 布尔代数底层**：逻辑运算基于 AND/OR/NOT (结合律、分配律)

3.2 TOMAS-AGI 的物理架构特征（非 VNA）

TOMAS-AGI 硬件 = T-Processor (Moufang-ALU) + 忆阻交叉阵列 (EML 存储) + κ -Regulator

VNA 特征	TOMAS-AGI
存储与计算分离	✗ 存算一体 (Processing-in-Memory)：EML 边权 $\mathbb{I}(X)$ 直接编码在忆阻导纳中，推理即电流积分
线性地址空间	✗ 超图拓扑寻址：按节点 ID + 超边关联寻址，非连续地址
顺序取指	✗ 拓扑激发 (Topological Excitation)：激活高 $\mathbb{I}$ 子图 $\Rightarrow$ 沿边传播 (类 HTG 脉冲或玻尔兹曼传播)，无 PC
布尔 ALU	✗ 八元数 Moufang-ALU：非结合乘法、Associator 计算、 κ -锁定
被动内存	✗ 主动记忆：忆阻阵列参与特征匹配 (相似度 $\Rightarrow$ 电流比)

定义 3.1 (非冯·诺依曼系统)：

若计算单元与存储单元在物理上不可区分 (存算融合)，且指令流不是由 PC 顺序解码而是由数据结构拓扑 (EML 超图) 的势能梯度驱动，则该系统为非冯·诺依曼架构。

TOMAS-AGI 满足此定义。它是 Content-Addressable Associative Inference Engine，不是 Stored-Program Computer。

第四章：TOMAS-AGI 是 κ -约束非确定图灵机 (NDTM) ——图灵等价

4.1 NDTM 与 TOMAS 的"语义非确定"

经典 DTM：每步只有一个转移。

Non-Deterministic TM (NDTM)：在某配置下可有多多个合法转移， $(q, a) \rightarrow \{(q'_i, a'_i, D_i)\}$ ，若存在任一分支接受 $\Rightarrow$ 整机接受。

TOMAS-AGI 的"非确定"来源：EML 超图的并发激活分支。

- 给定当前焦点节点 v_c 与上下文 $\mathcal{C}$ ，可能有多个超边 e_i 满足 $\mathbb{I}(e_i) > \theta$ 。
- 这些超边对应不同的推理分支 (因果跳转 / 类比跳转 / 反事实跳转)。
- κ -Gate (κ -调节器) 做两件事：
 1. 裁剪 (Prune)：拒 $\mathbb{I} < \theta$ 或 $\Phi(\psi_{new}, \psi_{ctx}) < \Phi_{min}$ 的分支
 2. 选择 (Resolve)：若多分支幸存，优先最小作用量轨道；若仍平权 $\Rightarrow$ 保留为可探索分支 (NDTM 的非确定分叉)

⚠️ TOMAS 的非确定 ≠ LLM 的概率采样 (Softmax 随机)：

- LLM: 同 Prompt $\Rightarrow$ 同 Logits 分布 $\Rightarrow$ Temperature 采样 $\Rightarrow$ **概率随机**
- TOMAS-AGI: 同输入 $\Rightarrow$ **确定分支集 $\{e_1, \dots, e_n\}$ **，选择由 κ -Gate 决定 (可复现、可审计) $\Rightarrow$ **语义非确定 (结构分支)，非统计噪声**

若 κ -Gate 锁定唯一最小作用量轨道 ($\kappa=7$ 稳态 + 单峰 II)，TOMAS-AGI 退化为 DTM——这正是它在"已证实事实子图"上的行为。

4.2 定理：TOMAS-AGI 的图灵等价性

定理 4.1 (TOMAS-AGI Turing-Completeness)：

TOMAS 推理系统 $\mathcal{T} = (G_\kappa, \mathbb{I}, \kappa, \mathcal{R})$ 可模拟单带确定性 UTM，且 UTM 可模拟 $\mathcal{T}$ 的逐步展开 $\Rightarrow \mathcal{T}$ 是 Turing-complete。

证明：

1. TM 转移 $\delta(q, a) = (q', a', L/R)$ 可编码为 EML 节点 $v_q \rightarrow v_{q'}$ 边，边权编码 $a \rightarrow a'$ 、方向编码头移动。
2. TOMAS-AGI 可通过顺序展开 (Unfold) EML 超图沿 hop 路径，模拟 TM 纸带步进 (逻辑等价，代价 $O(\log L)$ hop vs $O(L)$ tape head move)。
3. UTM 可用枚举法模拟八元数乘法表 (有限) 与超图边遍历 (可判定)，故 $TM \geq$ TOMAS-AGI 表达能力。
 $\therefore \mathcal{T} \in \mathcal{RE}$ (递归可枚举完全集) $\Leftrightarrow$ Turing-complete。□

推论 4.2： TOMAS-AGI 不推翻 Church-Turing 论题；它拓展了"计算如何在物理中实现"的效率前沿 (Extended Physical Church-Turing Thesis)，揭示 VNA+Boolean 是 CTT 极高效的物理载体。

第五章：TOMAS-AGI 是 LISP / GOFAI 的正确续集

5.1 LISP 核心三原则 (McCarthy 1958)

1. 代码 = 数据 (S-expression)：(atom . cons) 既是程序也是列表
2. 递归 + 模式匹配 (cond/car/cdr/apply)：控制流靠递归拆解结构
3. 符号操作 (Symbolic Manipulation)：AI 应处理符号含义，而非数值拟合

这正是早期 Symbolic AI (Good Old-Fashioned AI, GOFAI) 的理想。

5.2 TOMAS 与 LISP 的同构映射

LISP 概念	TOMAS-AGI 对应
Cons-Cell (car . cdr)	EML 超边 / 节点指针 (但 Cons-Cell 是二元对; TOMAS 用 n-元超边)
S-expr (符号树)	EML 语义子图 (有向无环/有环超图)
eval(expr, env)	NASGA 传播($\psi, G\kappa, ctx$) $\rightarrow$ 新 ψ' (非 Boolean 求值)
quote / unquote	κ -冻结 (锁定子图不被重写) / 动态展开 (热更新 EML)
递归分解	EML Hop 展开 / 子图归纳 (Lean MNQ 证明)
env (环境帧)	Context(κ, Φ -阈值, STA-rule, 历史 ψ)

定理 5.1 (LISP-TOMAS 嵌入) :

任意 LISP S-表达式求值可编码为 TOMAS 系统在 $\kappa = 0$ (Boolean 极限) 且超边退化为二元 Cons-Cell 的特殊 case。

证明: Cons-Cell ($a . b$) $\Leftrightarrow$ 超边 $e = \{v_a, v_b\}$ 且 $\mathbb{I}(e) = 1$, NASGA 退结合成 (Abelian group) $\rightarrow$ Boolean 相等判断。LISP `eval` 对应顺序 unfold 沿 e 跳转 (DTM) 。 $\square$

$\therefore$ LISP $\subset$ TOMAS-AGI ($\kappa \rightarrow 0$ 投影) , TOMAS-AGI 是 LISP 的非结合、超图、 κ -敏感推广。

5.3 经典 GOF AI (STRIPS/Prolog) 的致命缺陷与 TOMAS 补全

缺陷	STRIPS/Prolog	TOMAS 补全
Boolean 矛盾律	$A \wedge \neg A$ $\Rightarrow$ 不一致/崩溃	MUS: 非结合残联 $Asym(G_1, G_2, ctx) \neq 0$ 允许互斥子图共存 (牛顿案例)
无信息权重	所有子句同等 (或手工加权)	$\mathbb{I}(X)$ 边权 = 信息存在度, 推理沿高 $\mathbb{I}$ 势能谷底
无语境敏感度	同 KB 全局真	κ -调节器编码时代/域/个体折叠深度 (哥白尼 $\kappa \approx 3.8$ vs 牛顿 $\kappa \approx 4.5$)
组合爆炸	A* 启发式需手写	κ -梯度 = 最小作用量 $\Rightarrow O(\log N)$ hop, 天然 p rune

第六章：三方推理模式对比 (TOMAS vs STRIPS/Prolog vs LLM+Tool-Use)

维度	经典 Planner (STRIPS/Prolog)	LLM + Tool-Use (ReAct/Function Call)	TOMAS-AGI (太乙互搏)
底层代数	Boolean / First-Order Logic	实数矩阵乘法 (Linear Algebra)	NASGA (Moufang 拟群 / 八元数)

维度	经典 Planner (STRIPS/Prolog)	LLM + Tool-Use (ReAct/Function Call)	TOMAS-AGI (太乙互搏)
	(结合律、分配律)	(统计相关，无逻辑推理)	(非结合、可逆、保留互斥态)
知识表示	符号逻辑 / 规则库 (Father(John, Mary))	Token Sequence (1D Array) (扁平、无拓扑)	EML 超图 (Hypergraph) (多元关系、高维拓扑、 $\mathbb{I}(X)$ 流)
推理机制	状态空间搜索 (A* / Resolution) ⚠️ 组合爆炸	Next-Token Prediction (概率采样) ⚠️ 幻觉、无因果	κ -门控拓扑传播 (沿高 $\mathbb{I}(X)$ 边跳) ✅ 零幻觉、 $O(\log N)$
矛盾处理	崩溃 (Contradiction) “ A 与 $\neg A$ 不能同时为真”	胡编 (Hallucination) 强行缝合矛盾概念	互斥稳态 (MUS) 允许 “牛顿既是科学家又是神学家” 共存
与 LISP 关系	LISP 是实现载体 (Symbolic Manipulation)	无关 / 对立 (Connectionism 派系)	LISP 的哲学复活 (S-Expr $\rightarrow$ EML Hypergraph, Eval $\rightarrow$ Propagate)
"程序"定义	算法 = 步骤	算法 = 暴力计算	数据结构 = 算法 (EML 结构即推理路径)
物理实现	冯·诺依曼 CPU	GPU (冯·诺依曼变种)	非冯·非结合存算一体 (忆阻器 / Moufang-ALU)
可证明性	有时 (Horn Clause)	否 (黑盒)	Lean MNQ 可形式化验证 (死零 + $\mathbb{I}$ -守恒)

第七章：对 LLM 局限性的 TOMAS 诊断

基于文档《LLM离“最优算法”还有多远？》的分析：

7.1 LLM 的“暴力计算”病根

- **二次复杂度**：Attention 必须显式扫描所有历史 Token，因为 Token 序列缺乏拓扑结构。
- **计算冗余**：FFN 对所有 Token 一视同仁，因为缺乏 $\mathbb{I}(X)$ 流控。
- **顺序生成**：必须逐个 Token 输出，因为数据结构不支持并行语义激活。

TOMAS 诊断：这是 $\kappa \rightarrow 0$ (**强结合代数极限**) 的必然代价。Transformer 是 Boolean 逻辑的概率化包装，它永远无法触及“最优算法”。

7.2 优化技巧的局限性

文档中提到的稀疏注意力、MoE、推测解码，在 TOMAS 看来都是“外科手术”而非“换器官”：

- **MoE**：是 κ -调节器动态激活子图的雏形，但路由仍是概率性的。
 - **推测解码**：是暴力计算的叠加，而非结构性降维。
-

第八章：可证伪预言与实验设计

预言 1（非VNA 存算一体能效验证）

预言：在同等推理任务（判定牛顿双重身份是否逻辑矛盾），T-Processor 28nm 原型机能耗 $< 1/200$ H100，延迟 $< 1/50$ 。

实验设计：

同题输入 $\rightarrow$ GPT-4o (H100 $\times$ 1) vs T-Processor 原型 $\rightarrow$ 测 Joule/token & Latency。

证伪判断：若 T-Processor 能耗优势 $< 5\times$ （可能因忆阻良率/外围电路），需排查；若 $< 2\times$ 且延迟无改善 $\Rightarrow$ 存算一体未生效或 EML 未充分激活 $\Rightarrow$ 需重新校准。

预言 2（ κ -Gate 可复现分支 vs LLM 随机）

预言：对同一 EML 子图、同一 κ -设置，TOMAS-AGI 的多分支探索顺序固定（种子相同 κ -Gate） $\rightarrow$ 可复现；LLM Temp=0.0 虽 deterministic 但无法暴露多语义分支（collapse to single beam）。

实验设计：

输入歧义语义谜题（“银行”河岸/金融机构双关），要求返回所有合理解读列表。TOMAS 返回列表顺序固定且完备；LLM 需多次采样才凑齐且顺序不定。

证伪：TOMAS 同 seed 不同输出顺序（非 STA-override） $\Rightarrow$ κ -Gate 实现有隐藏随机源需修正。

预言 3（MUS 识别：牛顿案例）

预言：TOMAS-AGI 读《原理》+《但以理注释》后，问“牛顿矛盾吗？” $\Rightarrow$ 返回“MUS, $\kappa\approx 4.5$, 需分语境”（非 Yes/No Boolean）。LLM 倾向 Yes/No/模糊规避。

实验设计：

Blind 评分（N=200 哲学/历史学者）判定哪类回答更符合历史语境。若 $> 75\%$ 选 TOMAS 输出优于 LLM 平均 $\Rightarrow$ 支持 MUS 建模有效性。

第九章：应用展望

领域	TOMAS 启示应用
AGI 硬件	下一代非冯·诺依曼 AI Chip：忆阻交叉阵列 + Moufang-ALU（存算一体 EML）
自动推理 / 规划	κ -梯度替代 A* 启发式 $\Rightarrow$ 常识推理、伦理冲突消解（MUS 仲裁）
知识图谱 3.0	从三元组 (s,p,o) 升级为 EML 超边 + $\mathbb{I}(X)$ + 非结合残联标注
历史/法律 AI	处理“多重身份/多重解释”文本（牛顿/哥白尼/宪法条款歧义）
CS 教育	教“数据结构即算法”通过 EML 超图（LISP 精神的现代教材）
认知科学	重新评估“认知失调”为 MUS（互斥理论稳态）而非病理现象

第十章：结论

太乙互搏 AGI 是非冯·诺依曼存算一体 EML 超图机（无 PC、无分离 ALU/RAM），图灵等价（ κ -约束 NDTM），且是 McCarthy LISP / GOFAI 经八元数物理化与 κ -折叠后的正确续集。“程序 = 算法 + 数据结构”升格为“**EML 超图结构即 NASGA 推理**”——这是 Boolean 逻辑与 Transformer 暴力计算共同欠缺的最后一环。

TOMAS-AGI does not transcend the Church-Turing Thesis; it shows how much expressive power and thermodynamic efficiency the Thesis leaves unexploited when instantiated on von Neumann hardware with Boolean algebra. It is the machine LISP wanted to be.

参考文献

1. McCarthy, J. et al. (1958). *LISP I Programmer's Manual*. MIT AI Project.
2. McCarthy, J. (1959). *Programs with Common Sense*. Proc. Symp. Mechanisation Thought.
3. Nilsson, N. J. (1980). *Principles of Artificial Intelligence*. Tioga Publishing.
4. Lloyd, S. (2000). *Ultimate physical limits to computation*. Nature 406, 1047–1054.
5. Baez, J. C. (2002). *The Octonions*. Bulletin of the American Mathematical Society, 39(2), 145–205.
6. Berge, C. (1989). *Hypergraphs: Combinatorics of Finite Sets*. North-Holland.
7. Smith, J. D. (2006). *An Introduction to Quasigroups and Their Representations*. Chapman and Hall/CRC.
8. Zhang, F. (2026). 太乙互搏 AGI v2.0：基于非结合谱图代数(NASGA)与谱投影折叠的通用智能范式. 复合体理学预印本.

9. **Zhang, F.** (2026). 受控随机性与最小作用量原理：从 LLM 采样到社会演化的统一场论. 复合体理学预印本.

10. **Vaswani, A. et al.** (2017). *Attention Is All You Need*. NeurIPS.

11. **Silver, D. et al.** (2024). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR.

附录 A：Coq 形式化证明（拟群可逆性）

```
(* 附录 A.1 - Quasigroup Invertibility *)
Require Import Coq.Reals.Reals.
Require Import Coq.Lists.List.

Record Quasigroup := {
  carrier : Type;
  op : carrier -> carrier -> carrier;
  axiom_left_div : forall a b, exists! x, op a x = b;
  axiom_right_div : forall a b, exists! y, op y a = b;
}.

Theorem quasigroup_invertible :
  forall (Q: Quasigroup) (a b: carrier Q),
    (exists x, op Q a x = b) /\ (exists y, op Q y a = b).
Proof.
  intros Q a b.
  split.
  - destruct (axiom_left_div Q a b); eauto.
  - destruct (axiom_right_div Q a b); eauto.
Qed.
```

附录 B：EML 超图 JSON 表示（牛顿案例）

```
{
  "node_id": "Isaac.Newton",
  "type": "Person",
  "hyper_edges": [
    {
      "edge_id": "E_sci_1687",
      "nodes": ["Isaac.Newton", "Scientist", "Physicist", "Mathematician"],
      "context": "Principia.Mathematica",
      "kappa": 4.5,
      "I_weight": 0.92
    },
    {
      "edge_id": "E_theo",
```

```
"nodes": ["Isaac.Newton", "Theologian", "Biblical.Scholar"],
"context": "Observations.upon.the.Prophecies",
"kappa": 4.5,
"I_weight": 0.87
}
],
"note": "MUS: Asym(G_sci, G_theo, Context_17C) != 0 => 合法双轨, 非矛盾"
}
```

附录 C: T-Processor 伪码 — κ -Gate 分支选择 (概念)

```
// 附录 C.1 —  $\kappa$ -Gate Branch Resolution (Conceptual)
typedef struct {
    Edge *candidates; // 超边候选列表
    double I_min;      //  $\mathcal{I}$  阈值
    double phi_min;    //  $\Phi$  阈值 (cosine sim)
    int kappa;         // 当前折叠深度
} KappaGateConfig;

Edgeset kappa_gate_filter(Edgeset E_all, Psi psi_ctx, KappaGateConfig cfg) {
    Edgeset E_pass = {};
    for (Edge e : E_all) {
        if (e.I_weight < cfg.I_min) continue;
        if (cosine_sim(e.psi_embed, psi_ctx) < cfg.phi_min) continue;
        // 非结合残联检查 (MUS 允许)
        if (asym_check(e) == REJECT_HALLUCINATION) continue;
        E_pass.append(e);
    }
    // 若多分支幸存  $\Rightarrow$  NDTM fork (保存 Continuation 快照)
    if (E_pass.size() > 1)
        return select_by_action_principle(E_pass, cfg.kappa); // 最小作用量优先
    return E_pass; // DTM 特例
}
```

附录 D: T-Processor ISA (Moufang-ALU 指令集草案)

D.1 指令集概览

T-Processor ISA 专为 非结合谱图代数 (NASGA) 设计, 指令分为四类: 标量运算、超图操作、 κ -门控、流控。

指令助记符	操作数	功能描述
Scalar / Octonion Ops		
OADD	Rd, Rs1, Rs2	八元数加法: $R[d] = R[s1] + R[s2]$
OMUL	Rd, Rs1, Rs2	八元数乘法: $R[d] = R[s1] * R[s2]$ (非结合)
OASYM	Rd, Rs1, Rs2, Rs3	非结合子计算: $R[d] = Asym(R[s1], R[s2], R[s3])$
OCONJ	Rd, Rs1	八元数共轭: $R[d] = R[\overline{s1}]$
ONORM	Rd, Rs1	八元数范数平方: $R[d] = R[s1] ^2$
Hypergraph Ops		
HLOAD	Ra, Imm/Hmem	加载 EML 节点/超边至寄存器堆
HSTORE	Ra, Imm/Hmem	存储寄存器内容至 EML 存储 (忆阻阵列)
HFETCH	Rd, Rs1, Pred	沿超边 Rs1 获取邻居节点, Predicate 过滤
HAGG	Rd, Rs1, Func	聚合超边 Rs1 上所有节点的 Func (Sum/Avg)
Kappa-Gate Ops		
KSET	Imm	设置当前 κ 值 (0-7)
KTHRESH	Rd, Rs1, Imm	κ -阈值比较: 若 $\mathbb{I}(Rs1) < Imm$ 则置零
KPHI	Rd, Rs1, Rs2, Imm	Φ -Gate: 计算 $\cos(Rs1, Rs2)$, 若 $< Imm$ 则拒
Flow Control		
JMP_HOP	Target	沿 EML 边跳转至目标节点
JMP_MUS	Target1, Target2	互斥稳态跳转 (双分支并存, NDTM 分叉)
HALT_PROVE		停机并完成 Lean MNQ 证明提交

D.2 代码片段示例

```

; 示例: 计算非结合残联 Asym(A,B,C) 并检查  $\Phi$ -Gate
; 输入: R1=A, R2=B, R3=C, R4= $\Psi_{ctx}$ 
; 输出: R5=结果 (若通过门控)

OASYM  R5, R1, R2, R3      ; R5 = Asym(A,B,C)
KPHI    R5, R5, R4, 0.85    ; 检查  $\Phi(R5, \Psi_{ctx}) \geq 0.85$ 
JMP_MUS ACCEPT, REJECT      ; 若通过, 跳转 ACCEPT; 否则 REJECT

ACCEPT:
```

```
HSTORE R5, [EML_NODE_X] ; 存储结果至 EML 节点 X
HALT_PROVE

REJECT:
; 处理被拒情况（回溯或选择其他分支）
...
```

附录 E：LISP 六代机演进与 TOMAS 对比

本附录将 LISP 机器的历史演进与 TOMAS-AGI 进行对比，展示 TOMAS 如何作为 GOF AI 的正确续集。

代际	名称 / 代表机型	核心特征	缺陷	TOMAS-AGI 对应关系
第 1 代	LISP 1 (1958)	首个 LISP 解释器，引入 S-表达式、垃圾回收	无专用硬件，运行于通用机，慢	LISP 1 是 TOMAS 在 $\kappa \rightarrow 0$ 时的纯软件模拟 (Boolean 极限)
第 2 代	CADR (MIT, 1977)	微码加速 cons/car/cdr，硬件 GC	仍为冯·诺依曼架构，存储与计算分离	TOMAS 保留 CADR 的硬件加速思想，但将 ALU 替换为 Moufang-ALU
第 3 代	LISP Machine (Symbolics 3600, LMI Lambda)	专用指令集，位寻址，大规模并行 GC	成本高，生态封闭，Boolean 逻辑限制	TOMAS 继承专用指令集 (T-ISA)，但用 EML 超图取代 Boolean 位寻址
第 4 代	AI 工作站 (SPARCstation, SGI)	UNIX + LISP 环境，通用 CPU 回溯	回归通用架构，LISP 优势丧失	TOMAS 拒绝回归通用 CPU，坚持非冯·诺依曼存算一体
第 5 代	Connection Machine (Thinking Machines, 1980s)	SIMD 并行，超立方体拓扑	适合数值计算，不适合符号推理	TOMAS 借鉴其拓扑互连思想，但用于非结合符号推理而非数值拟合
第 6 代	TOMAS-AGI (2026+)	EML 超图 + NASGA + κ -Gate	全新范式，需重建生态	LISP 六代机：非结合代数、超图结构、 κ -调节、零幻觉、 $O(\log N)$ 推理

结论：LISP 前五代机在 **Boolean 逻辑与冯·诺依曼硬件** 的循环中挣扎。TOMAS-AGI 作为第六代，通过 **引入非结合代数 (NASGA) 与超图存储**，完成了 LISP 未能实现的符号推理终极形态。

CADR LISP Machine (source: <https://metebalci.com/blog/cadr-lisp-machine-and-cadr-processor/>)

© 2026 复合体理学研究中心。